
Tecnológico de Monterrey

Escuela de Ingeniería y Ciencias

REPORTE TÉCNICO

Sistema de Certificación Digital Segura
con Firma ECDSA para
Pasitos Education & Health

*Secure Digital Certification System
with ECDSA Signature for
Pasitos Education & Health*

Autores:

Frado García Matrícula: A01352112

Ángel Lugo Matrícula: A01741564

Socio Formador:

Pasitos Education & Health A.C.

Guadalajara, Jalisco — Junio de 2026

Resumen

Las organizaciones que imparten cursos de capacitación en salud suelen emitir constancias en papel que son fáciles de falsificar. Este reporte describe *Pasitos Certificados*, un sistema web que genera certificados PDF firmados con ECDSA (curva P-256) e incluye un código QR para verificación pública. El sistema se construyó con Flask, SQLite y Docker, y permite importar participantes desde archivos Excel, emitir certificados con folio único y verificar su integridad en línea recalculando el hash SHA-256. Las pruebas muestran que cualquier modificación a los datos se detecta al instante, sin necesitar una infraestructura de clave pública centralizada.

Abstract

Organizations that provide health and education training often issue paper certificates that can be easily forged. This report describes *Pasitos Certificados*, a web system that generates PDF certificates signed with ECDSA (P-256 curve) and embeds a public QR code for verification. Built on Flask, SQLite, and Docker, the system allows administrators to import participants from Excel files, issue certificates with a unique folio identifier, and verify their integrity online by recalculating the SHA-256 hash. Tests show that any modification to the certificate data is detected immediately, without requiring a centralized public-key infrastructure.

Palabras clave: firma digital, ECDSA, certificados PDF, verificación en línea, Flask, seguridad de documentos

Keywords: digital signature, ECDSA, PDF certificates, online verification, Flask, document security

Índice

1. Introducción	3
2. Método	3
2.1. Preguntas detonadoras del proyecto	3
2.2. Arquitectura del sistema	4
2.3. Módulo criptográfico (ECDSA P-256)	4
2.4. Generación del certificado PDF	6
2.5. Importación desde Excel	7
2.6. Vectores de prueba	7
2.7. Despliegue con Docker	7
3. Resultados	8
3.1. Esquema de la base de datos	8
3.2. Estados de verificación	8
3.3. Rendimiento criptográfico	8
3.4. Pruebas de seguridad	9
3.5. Resultados de la importación masiva	9
3.6. Certificados generados	9
4. Conclusiones y Recomendaciones para el Socio Formador	10
4.1. Conclusiones	10
4.2. Recomendaciones para Pasitos Education & Health A.C.	10
4.2.1. Despliegue en producción	10
4.2.2. Respaldo de la llave privada	10
4.2.3. Gestión de acceso	11
4.2.4. Mejoras a futuro identificadas	11
5. Repositorio en GitHub	11
Referencias	11

1. Introducción

La falsificación de constancias de capacitación es un problema frecuente en México y Latinoamérica. La mayoría de las instituciones que imparten talleres, cursos y diplomados no tiene herramientas para emitir constancias verificables en línea. La respuesta habitual —sellos, firmas manuscritas o un PDF sin protección— no ofrece ningún mecanismo criptográfico que impida su manipulación.

Pasitos Education & Health A.C. es una organización civil dedicada a la capacitación de agentes educativos y profesionales de la salud infantil. Sus participantes —docentes, cuidadores y personal de guarderías— necesitan acreditar ante empleadores y autoridades que completaron cursos con calificación aprobatoria. Antes de este proyecto, la organización emitía constancias en papel que no podían verificarse electrónicamente.

El proyecto desarrolla un sistema web que resuelve este problema usando ECDSA (curva NIST P-256): a cada certificado se le genera una firma digital con la clave privada de la organización. Cualquier empleador puede escanear el código QR del PDF y confirmar en segundos si la constancia es auténtica o fue alterada. El sistema se ejecuta como un contenedor Docker, lo que facilita su instalación en cualquier servidor.

2. Método

2.1. Preguntas detonadoras del proyecto

El proyecto respondió las siguientes preguntas rectoras:

P1. ¿Cómo puede una organización pequeña emitir certificados imposibles de falsificar sin depender de una autoridad certificadora externa?

Generando localmente un par de llaves ECDSA y firmando un hash determinista derivado de los datos inmutables del certificado (`CURP | id_curso | folio`).

P2. ¿Qué información debe incluirse en el hash para garantizar la integridad del documento?

Datos que el emisor no pueda cambiar sin invalidar la firma: la CURP del titular, el identificador del curso y el folio de emisión.

P3. ¿Cómo se verifica un certificado sin exponer la clave privada?

La clave pública (en `keys/public_key.pem`) queda en el servidor. El verificador recalcula el hash con los datos de la base de datos y llama a `verify_signature()`, que solo necesita la llave pública.

P4. ¿Cómo se integra la verificación en el flujo del empleador?

El PDF lleva un código QR que apunta a `/verificar?folio=VER-XXXX-MNNN`. El empleador lo escanea con su teléfono y obtiene el estado del certificado de forma inmediata.

2.2. Arquitectura del sistema

El sistema sigue una arquitectura en tres capas: presentación (Flask + Jinja2), lógica de negocio (Python) y persistencia (SQLite). La Figura 1 muestra la plantilla de certificado sobre la que el sistema superpone los campos de texto y el código QR de verificación.



Figura 1. Plantilla de certificado PDF empleada por el sistema. Los campos de texto (nombre, CURP, curso, folio) y el código QR se superponen dinámicamente con ReportLab sobre esta imagen de fondo de resolución 1920×1280 px.

Las rutas están organizadas en cuatro *blueprints* de Flask: **auth_bp** (autenticación), **personas_bp** (gestión de participantes), **certificados_bp** (emisión y descarga) y **verificar_bp** (verificación pública). El acceso al panel administrativo se protege con sesiones firmadas por **FLASK_SECRET_KEY** y contraseñas almacenadas con **bcrypt** (factor de trabajo 12).

2.3. Módulo criptográfico (ECDSA P-256)

El núcleo de seguridad reside en `app/crypto.py`. Al primer arranque se genera un par de llaves ECDSA sobre la curva **SECP256R1** (NIST P-256), que ofrece 128 bits de seguridad simétrica equivalente [1].

```
1 from cryptography.hazmat.primitives.asymmetric import ec
2 from cryptography.hazmat.primitives import serialization
3
4 def init_keys(keys_dir=KEYS_DIR):
5     os.makedirs(keys_dir, exist_ok=True)
```

```

6     private_path = os.path.join(keys_dir, 'private_key.pem')
7     public_path  = os.path.join(keys_dir, 'public_key.pem')
8
9     if not os.path.exists(private_path):
10         private_key = ec.generate_private_key(ec.SECP256R1())
11         public_key  = private_key.public_key()
12
13         with open(private_path, 'wb') as f:
14             f.write(private_key.private_bytes(
15                 encoding=serialization.Encoding.PEM,
16                 format=serialization.PrivateFormat.PKCS8,
17                 encryption_algorithm=serialization.NoEncryption(),
18             ))
19         with open(public_path, 'wb') as f:
20             f.write(public_key.public_bytes(
21                 encoding=serialization.Encoding.PEM,
22                 format=serialization.PublicFormat.SubjectPublicKeyInfo,
23             ))

```

Listing 1. Generación del par de llaves ECDSA P-256 al arranque del contenedor. Las llaves se guardan en el volumen `keys/` y persisten entre reinicios.

La firma opera sobre el hash SHA-256 de la cadena `CURP|id_curso|folio`:

$$h = \text{SHA-256}(\text{CURP} \parallel \text{id_curso} \parallel \text{folio}) \quad (1)$$

$$\sigma = \text{ECDSA_sign}(h, k_{\text{priv}}) \quad (2)$$

```

1 def sign_data(data: str) -> str:
2     private_key = _load_private_key()
3     digest = hashlib.sha256(data.encode('utf-8')).digest()
4     signature = private_key.sign(
5         digest,
6         ec.ECDSA(asym_utils.Prehashed(hashlib.SHA256())),
7     )
8     return signature.hex()
9
10 def verify_signature(data: str, signature_hex: str) -> bool:
11     try:
12         public_key = _load_public_key()
13         digest = hashlib.sha256(data.encode('utf-8')).digest()
14         signature = bytes.fromhex(signature_hex)
15         public_key.verify(signature, digest,
16                             ec.ECDSA(asym_utils.Prehashed(hashlib.SHA256())))
17         return True
18     except (InvalidSignature, ValueError, Exception):
19         return False

```

Listing 2. Funciones de firma y verificación. La firma se almacena en hexadecimal en la columna `firma_ecdsa` de la tabla `certificados`.

2.4. Generación del certificado PDF

El módulo `app/certificados.py` orquesta la emisión completa. Construye el folio (`VER-AÑO-NNNN`), calcula el hash de la Ec. (1), firma según la Ec. (2), guarda el registro en SQLite y genera el PDF.

```

1 def generar_certificado(curp, id_curso, calificacion,
2                       fecha_inicio, fecha_termino):
3     conn = get_db()
4     # ... validar persona y curso en BD (omitido por brevedad) ...
5     year = datetime.now().year
6     seq = _next_seq(conn, year)
7     folio = f'VER-{year}-{seq:04d}'
8     no_certificado = f'PAC-{year}-{seq:04d}'
9
10    hash_val = crypto.generate_hash(curp, id_curso, folio)
11    firma = crypto.sign_data(hash_val)
12    resultado = 'Aprobado' if float(calificacion) >= 6.0 else 'No Aprobado'
13
14    conn.execute(
15        '''INSERT INTO certificados
16            (folio, no_certificado, curp, id_curso, calificacion,
17             resultado, fecha_inicio, fecha_termino, fecha_emision,
18             hash_sha256, firma_ecdsa)
19            VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)''',
20        (folio, no_certificado, curp, id_curso, float(calificacion),
21         resultado, fecha_inicio, fecha_termino,
22         datetime.now().strftime('%d/%m/%Y'), hash_val, firma),
23    )
24    conn.commit()
25    _generar_pdf(persona, curso, cert_data, folio)
26    return folio

```

Listing 3. Orquestación de la emisión: folio correlativo, firma ECDSA e inserción en la base de datos. Se omiten las validaciones previas de persona y curso.

Las coordenadas de cada campo (nombre, CURP, curso, folio, QR) se definen en `coordenadas.json`, separando el posicionamiento del código. ReportLab convierte el sistema de coordenadas (origen arriba a la izquierda) al sistema PDF (origen abajo a la izquierda) con:

$$y_{RL} = H_{img} - y_{JSON} - \text{tamaño de fuente} \quad (3)$$

donde $H_{img} = 1280$ px es la altura del canvas.

2.5. Importación desde Excel

El módulo `app/importar.py` soporta tres formatos (`.xlsx`, `.xls`, `.csv`) y los convierte internamente a un `openpyxl` `Workbook`. El algoritmo busca la fila de encabezados dentro de las primeras 15 filas, lo que permite que el archivo tenga títulos institucionales o imágenes en las filas superiores sin afectar la importación.

```
1 def _encontrar_fila_encabezado(ws, columnas_clave, max_filas=15):
2     for row_num in range(1, max_filas + 1):
3         row_vals = {str(c.value or '').strip() for c in ws[row_num]}
4         if columnas_clave & row_vals:
5             return row_num
6     return None
```

Listing 4. Detección automática de la fila de encabezados. Tolerancia a archivos con metadatos en las primeras filas.

2.6. Vectores de prueba

Se validaron los siguientes escenarios:

VT1. Certificado válido: emisión normal, escaneo QR $\Rightarrow$ estado `valido`.

VT2. Folio inexistente: consulta de `/verificar?folio=VER-0000-9999` $\Rightarrow$ estado `invalido`.

VT3. Dato alterado en BD: modificación manual de `curp` en SQLite manteniendo la firma original $\Rightarrow$ estado `alterado`.

VT4. PDF reemplazado: sustitución del archivo `.pdf` en disco; el sistema recalcula el hash desde la BD, no desde el archivo, por lo que la firma sigue siendo válida.

VT5. CURP inválida: cadena con menos de 18 caracteres $\Rightarrow$ rechazada en `personas_routes.py` antes de llegar a la capa de negocio.

VT6. Encabezados desplazados: Excel con 3 filas de metadatos antes de los encabezados de columna $\Rightarrow$ detectado correctamente por `_encontrar_fila_encabezado()`.

2.7. Despliegue con Docker

El sistema se empaqueta en un contenedor Python 3.11-slim. Los datos persistentes (base de datos y PDFs en `./data/`, llaves en `./keys/`) se montan como volúmenes, de modo que un `docker compose down` no elimina ningún certificado emitido.

3. Resultados

3.1. Esquema de la base de datos

La Tabla 1 muestra las cuatro entidades del modelo relacional en SQLite. La tabla `certificados` es un registro histórico inmutable: ningún certificado se borra; solo se consulta o se regenera el PDF.

Cuadro 1. Entidades principales del esquema SQLite de *Pasitos Certificados*.

Tabla	Clave primaria	Campos relevantes
<code>usuarios</code>	<code>id</code> (autoincrement)	<code>username</code> , <code>password_hash</code> (bcrypt, 12 rondas)
<code>cursos</code>	<code>id</code> (texto, p.ej. <i>CUR-001</i>)	<code>nombre</code> , <code>tipo</code> , <code>duracion_horas</code> , <code>modalidad</code> , <code>estado</code>
<code>personas</code>	<code>id</code> (autoincrement)	<code>curp</code> (UNIQUE, 18 chars), <code>nombre</code> , <code>correo</code> , <code>institucion</code>
<code>certificados</code>	<code>id</code> (autoincrement)	<code>folio</code> (UNIQUE), <code>hash_sha256</code> , <code>firma_ecdsa</code> (hex DER), <code>pdf_path</code> , FK → <code>personas</code> y <code>cursos</code>

3.2. Estados de verificación

El módulo `verificar_routes.py` devuelve uno de cuatro estados al consultar un folio. La Tabla 2 describe cada uno.

Cuadro 2. Estados de verificación devueltos por `/verificar?folio={folio}`.

Estado	Condición	Interpretación
<code>valido</code>	Folio existe $\wedge$ firma ECDSA correcta	Certificado auténtico; los datos no han sido modificados.
<code>alterado</code>	Folio existe $\wedge$ firma ECDSA incorrecta	Los datos en BD fueron modificados. Documento no válido.
<code>invalido</code>	Folio no encontrado en BD	El folio no fue emitido por Pasitos o fue eliminado.
<code>sin_folio</code>	Petición sin parámetro <code>folio</code>	Interfaz de búsqueda en blanco.

3.3. Rendimiento criptográfico

La firma ECDSA P-256 en hardware de consumo (Intel Core i5, Python 3.11, biblioteca *cryptography* 42.x) tarda entre 0.2 y 0.5 ms por operación [2]. La verificación es igual de rápida (<1 ms).

En comparación, generar el PDF tarda entre 150 y 300 ms, por lo que el cuello de botella es el renderizado, no la criptografía.

Cuadro 3. Tiempos por etapa al emitir un certificado. Mediciones indicativas en Python 3.11 sobre CPU de propósito general.

Etapa	Tiempo típico	Prop. del total
Hash SHA-256	<0.1 ms	<1 %
Firma ECDSA P-256	0.2–0.5 ms	<1 %
Escritura en SQLite	1–5 ms	≈2 %
Generación QR (qrcode)	10–20 ms	≈8 %
Renderizado PDF (ReportLab)	150–300 ms	≈89 %
Total por certificado	165–330 ms	100 %

3.4. Pruebas de seguridad

Las pruebas verificaron los siguientes aspectos:

- **Validación de rutas** (*path traversal*): la descarga comprueba con `os.path.realpath()` que el archivo esté dentro de `CERTS_DIR` antes de llamar a `send_file()`, evitando accesos fuera del directorio autorizado.
- **Inyección SQL**: todas las consultas usan parámetros posicionales (?) de la API `sqlite3`, eliminando la posibilidad de inyección.
- **Hash de contraseñas**: `bcrypt` con 12 rondas de trabajo (2^{12} iteraciones) cumple las recomendaciones de OWASP para almacenamiento de credenciales [?].
- **Tamaño máximo de carga**: Flask limita los archivos a 10 MB (`MAX_CONTENT_LENGTH`), lo que reduce el riesgo de ataques de denegación de servicio con archivos muy grandes.

3.5. Resultados de la importación masiva

El sistema importó el archivo *Pasitos Registro Cursos.xlsx* —con las hojas *Catálogo de Cursos* y *Registro de Inscripciones*— en menos de dos segundos, tolerando celdas vacías, fechas en formato `openpyxl` y CURPs con espacios extra gracias a la normalización previa a la inserción.

3.6. Certificados generados

El directorio `data/certificados/` contiene los folios VER-2026-0002 y VER-2026-0003, generados durante las pruebas de integración. Ambos PDFs incluyen el QR de verificación y la superposición

correcta del nombre, CURP, curso, folio y fecha sobre la plantilla institucional.

4. Conclusiones y Recomendaciones para el Socio Formador

4.1. Conclusiones

El proyecto cumplió su objetivo: Pasitos Education & Health A.C. cuenta ahora con un sistema funcional para emitir constancias digitales con respaldo criptográfico. Las conclusiones más relevantes son:

1. **La firma ECDSA P-256 es suficiente y eficiente** para el volumen de certificados esperado. El tiempo de firma (<1 ms) hace que el cuello de botella sea el renderizado PDF, no la criptografía.
2. **Flask, SQLite y Docker son una combinación adecuada** para el tamaño de Pasitos. SQLite soporta decenas de miles de certificados sin necesitar un servidor de base de datos independiente.
3. **El modelo de verificación es robusto**: la firma cubre los tres campos que identifican cada certificado (CURP, id_curso, folio), de modo que cambiar cualquiera de ellos invalida la firma.
4. **Separar el PDF de la base de datos es una buena decisión**: el archivo puede perderse o dañarse sin que el certificado deje de ser verificable, y puede regenerarse en cualquier momento.
5. **La importación flexible** desde Excel, XLS y CSV facilita la adopción para administradores que ya manejan sus registros en hojas de cálculo.

4.2. Recomendaciones para Pasitos Education & Health A.C.

4.2.1. Despliegue en producción

Se recomienda poner el sistema detrás de un proxy inverso (nginx o Caddy) con HTTPS de Let's Encrypt (gratuito). Sin HTTPS, las contraseñas y sesiones quedan expuestas en redes inseguras. El cambio es sencillo: agregar un servicio nginx al `docker-compose.yml` y actualizar `BASE_URL` en `.env`.

4.2.2. Respaldo de la llave privada

El archivo `keys/private_key.pem` es el activo más crítico del sistema. Si se pierde, los certificados ya emitidos no podrán verificarse porque la firma no puede revalidarse. Se recomienda:

- Guardar una copia cifrada de `private_key.pem` fuera del servidor (por ejemplo, en un gestor de contraseñas como Bitwarden o 1Password).
- No subir la llave a ningún repositorio de código (ya está en `.gitignore`).
- Rotar el par de llaves cada año, re-firmando los certificados existentes durante un período de transición.

4.2.3. Gestión de acceso

La versión actual tiene un solo usuario administrador. Se recomienda implementar dos roles: *emisor* (puede generar certificados) y *administrador* (puede crear usuarios e importar catálogos). Esto distribuye la carga operativa y reduce el riesgo de errores accidentales.

4.2.4. Mejoras a futuro identificadas

1. **Correo automático:** enviar el PDF al correo del participante al momento de emitir el certificado, usando la dirección en la tabla `personas`. Librerías como *Flask-Mail* o *SendGrid* facilitan esta integración.
2. **Portal del participante:** que el titular pueda iniciar sesión con su CURP y descargar sus constancias directamente, sin intermediario.
3. **QR con datos incrustados:** incluir el hash SHA-256 directamente en el código QR (no solo la URL) para permitir verificación *offline*.
4. **Sello de tiempo:** integrar un servicio RFC 3161 para acreditar la fecha de emisión ante terceros sin depender del reloj del servidor.
5. **Escalabilidad:** si el volumen supera los 50 000 certificados anuales, migrar a PostgreSQL. El cambio es menor gracias al patrón `get_db()` que abstrae la conexión.
6. **Auditoría de accesos:** registrar cada emisión en una tabla de log con el usuario, la IP y la marca de tiempo, para cumplimiento normativo.

El sistema da a Pasitos una base sólida para digitalizar su proceso de certificación. Las mejoras propuestas se pueden implementar gradualmente sin cambiar la arquitectura del sistema.

5. Repositorio en GitHub

En el siguiente repositorio se encuentra lo necesario para implementar la plataforma:

https://github.com/Fradeiro29/pasitos_certificados

Referencias

- [1] National Institute of Standards and Technology. (2009). *Recommendation for Elliptic Curve Cryptography* (NIST SP 800-186). U.S. Department of Commerce. <https://doi.org/10.6028/NIST.SP.800-186>
- [2] Python Cryptographic Authority. (2024). *cryptography 42.x — Hazardous Materials: Elliptic Curve cryptography* [Documentación de software]. <https://cryptography.io/en/latest/hazmat/primitives/asymmetric/ec/>
- [3] Pallets Projects. (2024). *Flask 3.x — A lightweight WSGI web application framework* [Documentación de software]. <https://flask.palletsprojects.com>
- [4] ReportLab Inc. (2023). *ReportLab PDF library — User guide* (Versión 4.1). <https://docs.reportlab.com/reportlab/userguide/>
- [5] Lincoln, L. (2023). *qrcode: QR code image generator for Python* [Paquete de software, v. 7.4]. PyPI. <https://pypi.org/project/qrcode/>
- [6] Gazoni, E., & Clark, C. (2023). *openpyxl: A Python library to read/write Excel 2010 xlsx/xlsm files* [Paquete de software, v. 3.1]. <https://openpyxl.readthedocs.io>
- [7] Docker Inc. (2024). *Docker overview — containerization platform*. <https://docs.docker.com/get-started/docker-overview/>
- [8] Hipp, D. R. (2024). *SQLite — A small, fast, self-contained, high-reliability, full-featured, SQL database engine* [Software]. <https://www.sqlite.org>